INT315 · CLUSTER COMPUTING

UNIT III

Using RDDs for Creating Applications in Spark and Graph Analytics with Spark GraphX

RDDs, Shared Variables, the Spark Shell, GraphX & Feature Engineering

Theory -> Comic Visuals -> Industry Context -> Commands/Code -> Self-Assessment

Ten syllabus topics, in full depth, with heavy runnable code and seven original diagrams

Session 2026-27

Companion Study Material — INT315 Cluster Computing, Unit III

Table of Contents

Unit III at a Glance

Units I and II built the two foundations this unit finally puts to work: Unit I explained WHY Spark exists and what an RDD conceptually is (an immutable, partitioned, lineage-tracked collection); Unit II gave you the Scala language — val/var, collections, higher-order functions, currying — that Spark's RDD API is written in and that you'll use to actually program against it. This unit is where those two threads merge: you'll create real RDDs, transform and act on them using the exact higher-order-function style from Unit II Section 6, run real distributed jobs from an interactive shell, share state safely across a cluster, and then pivot to an entirely new capability — graph-parallel computation with Spark GraphX — closing with the feature engineering techniques that turn raw data into the numeric vectors machine learning algorithms actually consume.

This is the most code-heavy unit yet, and deliberately so: RDDs and GraphX are both APIs, and APIs are learned by writing code against them, not by reading about them. Every topic below still follows the five-beat rhythm — Theory & Mechanics, a Comic/Visual metaphor, Industry Context, Commands/Code, and Self-Assessment — but expect longer, denser code blocks throughout, exactly as requested.

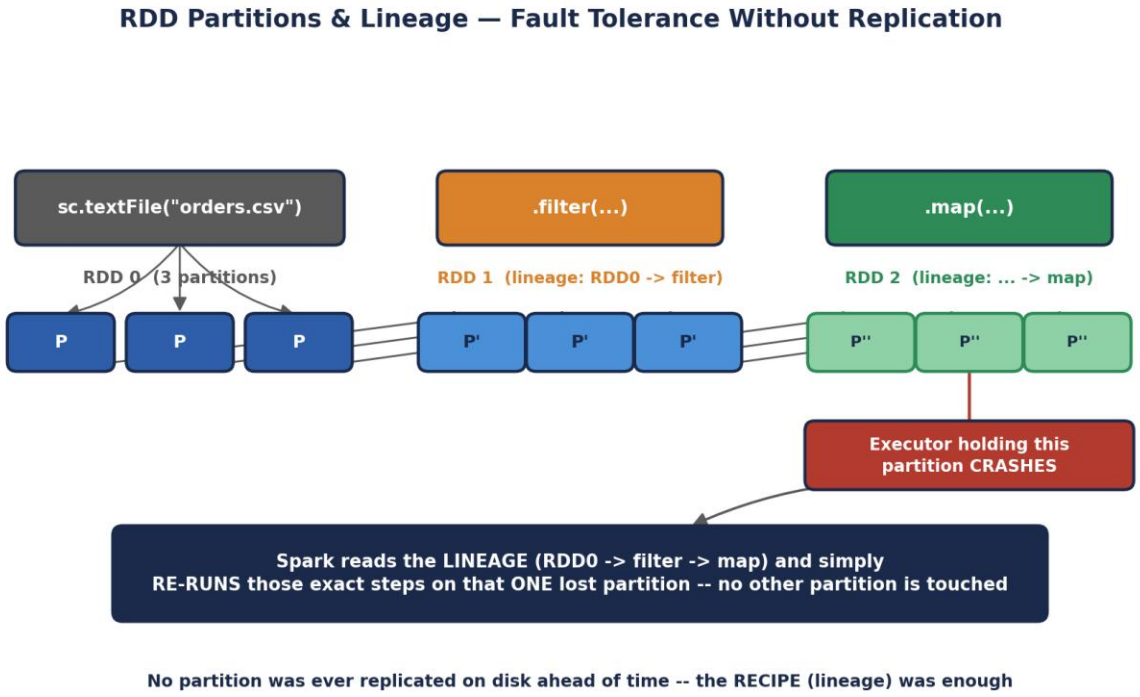
> **From Blueprint to Actual Construction**
>
> *If Unit I was the architect's pitch for why a new building technique (Spark) beats the old one, and Unit II was mastering the specific tools (Scala) the construction crew uses, Unit III is the day you actually pick up those tools and start building -- pouring the foundation (creating RDDs), framing the structure (transformations and actions), wiring the electrical and plumbing that lets different parts of the building share resources (shared variables), and finally adding an entirely new wing built from a different set of blueprints (GraphX) for a fundamentally different kind of problem: not rows of data, but networks of relationships.*

1. Features of RDD

An RDD (Resilient Distributed Dataset) is Spark's foundational data structure — Unit I introduced the term; this section unpacks EVERY word in that name precisely, because each word corresponds to a specific, deliberate design feature.

1.1 Resilient — Fault Tolerance Through Lineage

"Resilient" refers specifically to how an RDD survives node failure. Rather than replicating every partition to disk ahead of time (HDFS's approach, per earlier units), Spark tracks the LINEAGE of each RDD -- the exact sequence of transformations that produced it from its original source. If a partition is lost because an Executor crashes, Spark doesn't need a backup copy sitting on disk; it simply re-runs the recorded chain of transformations for JUST that lost partition.



RDD Partitions & Lineage — Fault Tolerance Without Replication

```
val raw = sc.textFile("hdfs:///data/orders.csv")   // RDD 0
val filtered = raw.filter(line => line.contains("COMPLETED"))  // RDD 1
val parsed = filtered.map(line => line.split(","))             // RDD 2

// Ask Spark to show you the lineage graph directly
println(parsed.toDebugString)
```

```
// (2) MapPartitionsRDD[2] at map
//  |  MapPartitionsRDD[1] at filter
//  |  hdfs:///data/orders.csv MapPartitionsRDD[0] at textFile
```

1.2 Distributed — Partitioned Across the Cluster

Every RDD is split into PARTITIONS, each of which lives on (and is processed by) one Executor. This is the same partitioning concept from earlier units, now applied concretely: an RDD isn't one big blob of data sitting in one place, it's a collection of smaller chunks spread across the cluster, processed in parallel.

```
val nums = sc.parallelize(1 to 100, 4)   // explicitly request 4 partitions
println(nums.getNumPartitions)           // 4

// See exactly what's in each partition
nums.glom().collect().foreach(part => println(part.mkString(",")))
// prints 4 lines, one per partition, each showing ~25 numbers

// Repartitioning -- changes the NUMBER of partitions (triggers a shuffle)
val repartitioned = nums.repartition(8)
println(repartitioned.getNumPartitions)  // 8
```

1.3 Dataset — Immutable, Type-Safe Collections

Directly inheriting Unit II Section 5's Scala philosophy, an RDD is IMMUTABLE: once created, its contents never change. Every transformation returns a brand-new RDD, exactly like a Scala List's :+ returns a brand-new List. RDDs are also TYPED — an RDD[Int] and an RDD[String] are genuinely different types the Scala compiler checks at compile time, per Unit II Section 1.3's static typing feature.

```
val original: RDD[Int] = sc.parallelize(List(1, 2, 3))
val doubled: RDD[Int] = original.map(_ * 2)

// original is completely untouched -- exactly like Scala's immutable List
println(original.collect().toList)   // List(1, 2, 3)
println(doubled.collect().toList)    // List(2, 4, 6)
```

1.4 Lazy Evaluation

As introduced in Unit I Section 4.4, RDD transformations are lazy -- calling .map() or .filter() only builds up the lineage graph; nothing computes until an ACTION (like .collect() or .count()) is called. This is worth re-anchoring specifically for RDDs here because it directly explains why .toDebugString above can show a full multi-step lineage graph without ever having touched the actual 100-million-row file on disk.

1.5 In-Memory Caching / Persistence

An RDD can be explicitly told to stay resident in Executor memory across multiple actions, rather than being recomputed from its lineage every single time it's used. This is the direct, hands-on version of Unit I's "in-memory processing" advantage over MapReduce.

```
val expensive = sc.textFile("hdfs:///data/huge.csv")
                  .map(line => expensiveParse(line))

expensive.cache()   // mark for in-memory caching (shorthand for
persist(MEMORY_ONLY))

val count1 = expensive.count()     // computes AND caches the result
val count2 = expensive.filter(_.nonEmpty).count()   // reuses the CACHED data --
                                                      // does NOT re-run
expensiveParse

import org.apache.spark.storage.StorageLevel
expensive.persist(StorageLevel.MEMORY_AND_DISK)   // spill to disk if memory
runs out
expensive.unpersist()   // explicitly free the cached data when done with it
```

> ⚠ **A subtlety worth catching early**
>
> Calling .cache() or .persist() does NOT itself trigger computation — it's still just a marker on the RDD, respecting Section 1.4's laziness. The actual caching happens the FIRST time an action forces that RDD to be computed; every action after that reuses the cached partitions instead of recomputing from lineage.

1.6 Coarse-Grained Transformations

RDD operations are deliberately COARSE-GRAINED: a transformation like .map(f) applies the SAME function f to every element of the dataset, rather than allowing arbitrary, element-specific modifications. This constraint is precisely what makes lineage-based fault tolerance (Section 1.1) cheap to record and cheap to replay — Spark only needs to remember "apply this one function to this data," not a long, arbitrary log of millions of individual edits.

> **A Recipe Card, Not a Diary**
>
> *Coarse-grained lineage is a RECIPE CARD: "take the flour, sift it, fold in the eggs" -- a short, general instruction applied uniformly to the whole bowl. Fine-grained mutation would be a DIARY: "at 3:01pm I picked up grain #4,502 and moved it two inches left; at 3:02pm I picked up grain #4,503..." -- a step-by-step log of every individual change.*

If the bowl falls on the floor (a partition is lost), replaying a recipe card takes a moment. Replaying a diary of millions of individual grain-level edits would take forever and cost enormous memory just to STORE the diary in the first place. RDDs are deliberately designed to only ever need recipe cards.

Why 'Resilient Distributed Dataset' is a precise engineering name, not marketing

Each of the three words in the name maps to a concrete design decision with a real production consequence: Resilient (lineage) means no data is ever silently lost to a single node failure without automatic recovery; Distributed (partitioning) means the dataset scales horizontally across as many machines as the cluster provides; Dataset (immutable, typed) means the compiler catches type errors before a multi-hour job ever runs, and no task can ever corrupt data another task is relying on. Interviewers who ask "what does RDD stand for, and why does it matter" are testing exactly this — not rote memorization, but whether you understand each word as an engineering trade-off.

Check yourself — Section 1

Q1. Explain, in one sentence each, what "Resilient," "Distributed," and "Dataset" specifically mean for an RDD.

Q2. Why does calling `.cache()` on an RDD not immediately load anything into memory?

Q3. What specific property of RDD transformations (mentioned in Section 1.6) makes lineage-based fault tolerance cheap?

Hands-on Challenge:

Create an RDD from a range of 1 to 1000 with 5 partitions, chain at least three transformations onto it, call `.toDebugString` to view the lineage BEFORE calling any action, then call `.count()` and explain in your own words what happens at that exact moment that didn't happen before.

2. Creating RDDs

There are three distinct ways to create an RDD, and understanding all three -- and when each is appropriate -- is a core practical skill for the rest of this unit.

2.1 Parallelizing an Existing Scala Collection

The sc.parallelize() method takes an existing in-memory Scala collection (built using Unit II's List, Vector, Array, and similar) and distributes it across the cluster as an RDD. This is the most common way to create small, illustrative RDDs for testing and learning.

```
// From a simple List (Unit II Section 7.1)
val numbersRDD = sc.parallelize(List(10, 20, 30, 40, 50))

// Explicitly controlling the number of partitions
val numbersRDD4 = sc.parallelize(List(10, 20, 30, 40, 50), numSlices = 4)

// From a Range
val rangeRDD = sc.parallelize(1 to 1000000)

// From a Scala Map -- creates an RDD of (key, value) TUPLES
val salaryRDD = sc.parallelize(Map("Aditi" -> 85000, "Rahul" -> 72000).toSeq)
println(salaryRDD.collect().toList)
// List((Aditi,85000), (Rahul,72000))
```

When to actually use parallelize()

In real production Spark applications, `parallelize()` is used sparingly — mainly for testing, small lookup tables, or seeding an initial dataset. Its fundamental limitation is that the ENTIRE collection must already exist in the Driver's memory before it can be distributed, which defeats the purpose for genuinely large datasets. Section 2.2's `textFile()` is the far more common real-world entry point.

2.2 Loading from an External Data Source

The far more common real-world path: loading data that ALREADY lives in a distributed storage system (HDFS, S3, or the local filesystem), so it never needs to pass through the Driver's memory at all -- each Executor reads its own portion directly.

```
// Plain text file -- one RDD element per LINE
val lines = sc.textFile("hdfs:///data/access.log")

// A whole directory of files at once (wildcard supported)
val allLogs = sc.textFile("hdfs:///data/logs/*.log")
```

```
// wholeTextFiles -- one element per FILE, as (filename, full content) pairs
// useful when file boundaries themselves carry meaning
val wholeFiles = sc.wholeTextFiles("hdfs:///data/documents/")
wholeFiles.foreach { case (filename, content) =>
  println(s"$filename has ${content.length} characters")
}

// Reading from the LOCAL filesystem (single-machine testing)
val localData = sc.textFile("file:///home/student/sample.txt")

// Structured formats (used far more heavily once Spark SQL is introduced,
// but accessible from the RDD API too)
val sequenceData = sc.sequenceFile[String, Int]("hdfs:///data/pairs.seq")
```

> ⚠ **A very common beginner error**
>
> `sc.textFile("data.csv")` with a bare relative path, when running on an actual cluster, will fail with a file-not-found error on every Executor that doesn't have that exact local file — because each Executor independently tries to resolve that path on ITS OWN machine. In cluster mode, always use a fully-qualified path (hdfs://, s3://, or an explicitly shared filesystem path) rather than a bare local filename, except when deliberately running in local mode for testing.

2.3 Transforming an Existing RDD

The third path isn't really a separate mechanism — it's simply Section 1.4's lazy transformations, which is worth stating explicitly here because it means EVERY transformation you write throughout this unit is, technically, also "creating an RDD." map(), filter(), and every other transformation covered in Section 3 all create a brand-new RDD from an existing one.

```
val original = sc.parallelize(1 to 10)
val evens = original.filter(_ % 2 == 0)      // a NEW RDD, created FROM original
val doubled = evens.map(_ * 2)               // another NEW RDD, created FROM
evens

// original, evens, and doubled are three DIFFERENT, independent RDD objects,
// each remembering its own place in the lineage chain
```

> **Three Doors Into the Same Room**
>
> *Think of an RDD's existence as a room you can walk into through three different doors. Door 1 (parallelize) is walking in carrying a box of things you already had in your hands. Door 2 (textFile / external sources) is a room that was ALREADY stocked with material by someone else (HDFS, S3) before you ever arrived -- you just walk in and start working with what's there. Door 3 (transformations) isn't really a separate door at all -- it's rearranging furniture that's already in a room you're standing in,*

which happens to create a brand-new room next door with the rearranged layout, while the original room stays exactly as it was.

Which path real pipelines actually use

Production Spark applications overwhelmingly enter through Section 2.2's external-source path — reading from a data lake (S3, HDFS, or a cloud warehouse) — precisely because real datasets are far too large to first collect into Driver memory the way `parallelize()` requires. `parallelize()` shows up constantly in unit tests and tutorials (including this one) specifically because it's the fastest way to create a small, deterministic RDD for demonstrating a concept — not because it's how production pipelines actually begin.

Check yourself — Section 2

Q1. Why is `sc.parallelize()` a poor choice for loading a 500GB dataset, even though it technically works?

Q2. What's the practical difference between `sc.textFile()` and `sc.wholeTextFiles()`?

Q3. Why does calling `.filter()` on an RDD count as "creating an RDD" in the same sense as `sc.parallelize()` does?

Hands-on Challenge:

Create one RDD using `sc.parallelize()` from a Scala List of at least 10 strings, and — if you have access to any text file locally or on HDFS — create a second RDD using `sc.textFile()`. Print the number of partitions and the first 3 elements of each using `.take(3)`.

3. RDD Functions

This section catalogs the TRANSFORMATION functions available on RDDs -- the lazy, DAG-building operations from Section 1.4. Every one of these takes an existing RDD and returns a brand-new one, following Section 1.3's immutability, and every function that accepts a Scala function argument uses exactly the higher-order-function syntax from Unit II Section 6.3.

3.1 Element-Wise Transformations

Function	Signature	What It Does
`map`	`(A => B) => RDD[B]`	Applies a function to every element, 1-to-1, producing a new RDD of possibly a new type
`flatMap`	`(A => Seq[B]) => RDD[B]`	Like map, but each input can produce ZERO, ONE, or MANY outputs, all flattened into one RDD
`filter`	`(A => Boolean) => RDD[A]`	Keeps only elements for which the predicate returns true
`distinct`	`() => RDD[A]`	Removes duplicate elements (requires a shuffle — Section 4)
`mapPartitions`	`(Iterator[A] => Iterator[B]) => RDD[B]`	Like map, but operates on a WHOLE partition at once — useful for expensive per-partition setup

```
val words = sc.parallelize(List("spark is fast", "scala is elegant"))

// map: one line in, one element out
val upper = words.map(_.toUpperCase)
println(upper.collect().toList)
// List(SPARK IS FAST, SCALA IS ELEGANT)

// flatMap: one line in, MANY words out, all flattened into a single RDD
val allWords = words.flatMap(line => line.split(" "))
println(allWords.collect().toList)
// List(spark, is, fast, scala, is, elegant)

// filter: keep only words longer than 4 characters
println(allWords.filter(_.length > 4).collect().toList)
// List(spark, scala, elegant)

// distinct: remove duplicates -- "is" appears twice above, once after distinct
println(allWords.distinct().collect().toList)
// List(spark, is, fast, scala, elegant)   -- order not guaranteed

// mapPartitions: set up an expensive resource ONCE per partition, not once per
element
```

Function	Signature	What It Does

```
val processed = allWords.mapPartitions { iter =>
  val expensiveResource = new java.util.Random(42)  // created ONCE per
partition
  iter.map(word => (word, expensiveResource.nextInt(100)))
}
```

3.2 Set-Like Operations Between Two RDDs

Function	Signature	What It Does
union	RDD[A] => RDD[A]	Combines two RDDs into one (keeps duplicates, no shuffle needed)
intersection	RDD[A] => RDD[A]	Elements present in BOTH RDDs (requires a shuffle)
subtract	RDD[A] => RDD[A]	Elements in this RDD that are NOT in the other (requires a shuffle)
cartesian	RDD[B] => RDD[(A,B)]	Every possible pair between the two RDDs — expensive, grows multiplicatively

```
val setA = sc.parallelize(List(1, 2, 3, 4))
val setB = sc.parallelize(List(3, 4, 5, 6))

println(setA.union(setB).collect().sorted.toList)
// List(1, 2, 3, 3, 4, 4, 5, 6)  -- duplicates kept

println(setA.intersection(setB).collect().sorted.toList)
// List(3, 4)

println(setA.subtract(setB).collect().sorted.toList)
// List(1, 2)   -- in setA, not in setB

val small1 = sc.parallelize(List("X", "Y"))
val small2 = sc.parallelize(List(1, 2))
println(small1.cartesian(small2).collect().toList)
// List((X,1), (X,2), (Y,1), (Y,2))  -- all 4 combinations
```

3.3 Sampling

```
val big = sc.parallelize(1 to 1000000)

// sample(withReplacement, fraction, seed) -- a random subset
val tenPercent = big.sample(withReplacement = false, fraction = 0.1, seed = 42)
```

```
println(tenPercent.count())   // approximately 100,000 (not exact --
probabilistic)
```

3.4 Pair RDD Transformations — Working with Key-Value Data

When an RDD's elements are TUPLES of the form (key, value), Spark unlocks an entire additional family of transformations specifically for key-value data — directly parallel to Unit II Section 7.2's Map operations, but operating across a distributed dataset.

Function	Signature	What It Does
groupByKey	RDD[(K,V)] => RDD[(K,Iterable[V])]	Groups all values sharing a key together (a wide, shuffle-heavy transformation)
reduceByKey	((V,V)=>V) => RDD[(K,V)]	Combines values per key using a function — pre-aggregates locally BEFORE shuffling
mapValues	(V=>W) => RDD[(K,W)]	Applies a function to just the VALUE half of each pair, leaving keys untouched
sortByKey	() => RDD[(K,V)]	Sorts the RDD by key (requires K to have an implicit Ordering)
join	RDD[(K,W)] => RDD[(K,(V,W))]	Inner join — pairs matching keys from two Pair RDDs together

```
val sales = sc.parallelize(List(
  ("Electronics", 500), ("Grocery", 120), ("Electronics", 300), ("Grocery", 80)
))

// reduceByKey -- the PREFERRED way to aggregate by key (see the warning below)
val totals = sales.reduceByKey((a, b) => a + b)
println(totals.collect().toList)
// List((Electronics,800), (Grocery,200))

// mapValues -- transform values, keys untouched, no shuffle needed
val withTax = sales.mapValues(amount => amount * 1.18)

// sortByKey
println(totals.sortByKey().collect().toList)
// List((Electronics,800), (Grocery,200))  -- alphabetical by key

// join -- combining two pair RDDs on a shared key
val categoryManagers = sc.parallelize(List(
  ("Electronics", "Rahul"), ("Grocery", "Meera")
))
println(totals.join(categoryManagers).collect().toList)
// List((Electronics,(800,Rahul)), (Grocery,(200,Meera)))
```

Function	Signature	What It Does

⚠ groupByKey vs. reduceByKey — a classic, real performance trap

`groupByKey` shuffles EVERY value across the network first, and only combines them afterward — for the sales example, all 500 and 300 travel across the network as separate values before being summed. `reduceByKey` combines values LOCALLY on each partition FIRST (exactly like MapReduce's Combiner from earlier units), then shuffles only the much smaller partial sums. For any associative, combinable aggregation, `reduceByKey` is almost always dramatically more efficient, and this exact distinction is one of the most common Spark performance-tuning interview questions.

The Post Office Sorting Room

map is a single postal worker stamping every letter with today's date, one at a time, one stamp per letter -- straightforward, 1-to-1. flatMap is a worker who opens each envelope and might pull out ZERO letters (junk mail, discarded), ONE letter, or a whole bundle of letters stuffed inside one envelope -- and dumps everything into one big outgoing pile, flattened.

groupByKey is naively hauling every single letter addressed to each city all the way to that city's sorting room before doing ANY counting -- enormous, wasteful cross-country shipping. reduceByKey is smart: it has each LOCAL post office pre-count and bundle its own city-bound letters into ONE summary parcel first, and only THEN ships the much smaller summary parcels cross-country -- the exact same final count, at a fraction of the shipping cost.

Where these functions show up constantly in real pipelines

`reduceByKey` and `join` are two of the most common operations in real ETL pipelines — computing daily revenue totals per product category, or joining a click-stream RDD against a user-profile RDD by user ID. `mapPartitions` shows up specifically in pipelines that need per-partition resources — a database connection, a loaded ML model, a compiled regex — that would be wastefully expensive to create fresh for every single row via a plain `map`.

Check yourself — Section 3

Q1. What's the difference between `map` and `flatMap`, and give an example where `flatMap` is clearly the right choice.

Q2. Why is `reduceByKey` generally preferred over `groupByKey` for a sum-style aggregation?

Q3. What does `cartesian` compute, and why should it be used carefully on large RDDs?

Hands-on Challenge:

Given a Pair RDD of (product, price) tuples with at least 8 entries across 3 products, use `reduceByKey` to compute the total price per product, `mapValues` to apply a 10% discount to each total, and `sortByKey` to present the final results alphabetically. Print the result at each stage.

4. Describe RDD Operations and Methods

Section 3 covered TRANSFORMATIONS -- lazy, DAG-building operations. This section covers ACTIONS -- the operations that actually trigger computation (Section 1.4), plus the crucial narrow-vs-wide distinction that determines how expensive a chain of transformations really is.

4.1 Actions — What Actually Triggers Computation

Action	Returns	What It Does
`collect`	`Array[A]`	Brings EVERY element back to the Driver — dangerous on large RDDs (can crash the Driver with OOM)
`count`	`Long`	Returns the number of elements
`first`	`A`	Returns just the first element
`take(n)`	`Array[A]`	Returns the first n elements — safe on large RDDs, unlike collect
`reduce`	`A`	Combines all elements into one, using an associative function
`foreach`	`Unit`	Applies a function to each element for its SIDE EFFECTS (e.g. writing to a database)
`saveAsTextFile`	`Unit`	Writes the RDD to a distributed file system, one partition per output file
`countByValue`	`Map[A,Long]`	Counts occurrences of each distinct value — brings the (usually small) result to the Driver

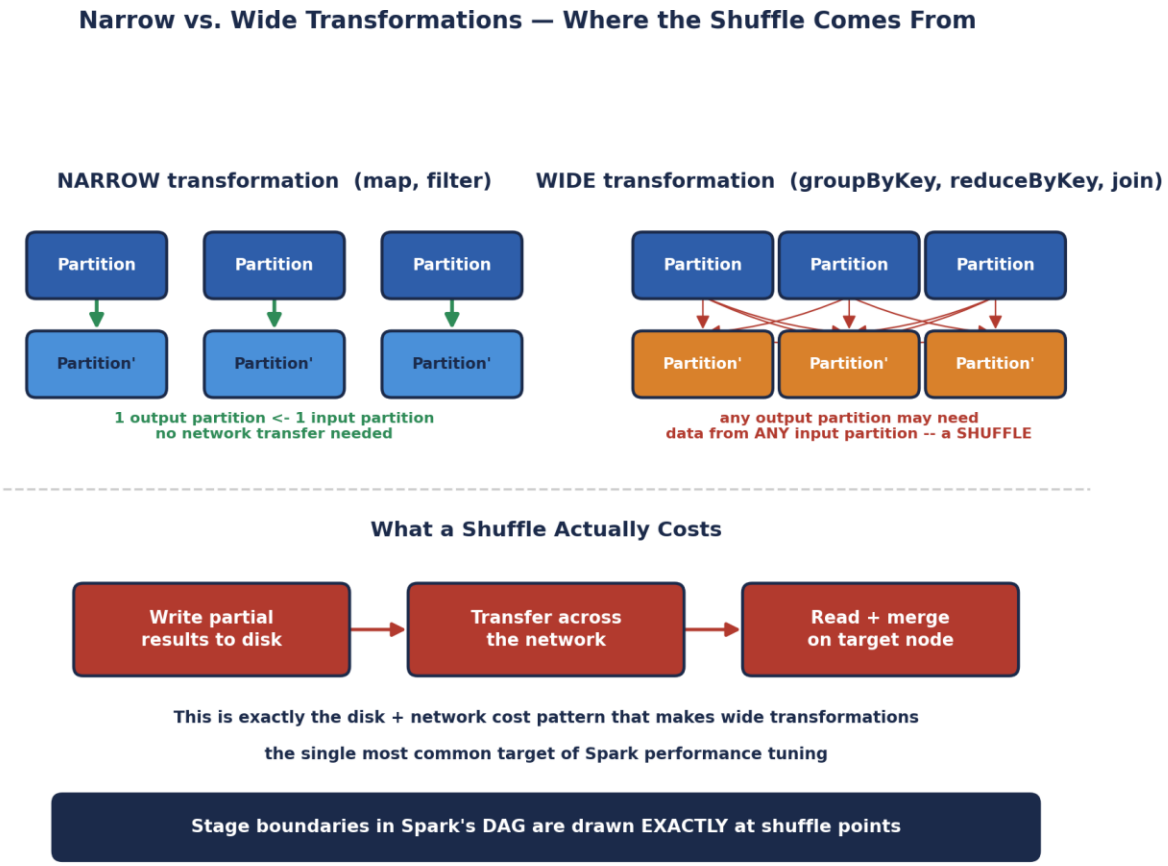
```
val nums = sc.parallelize(1 to 20, 4)

println(nums.count())            // 20
println(nums.first())            // 1
println(nums.take(5).toList)     // List(1, 2, 3, 4, 5)
println(nums.reduce((a, b) => a + b))   // 210

nums.foreach(n => if (n % 7 == 0) println(s"$n is divisible by 7"))
// runs ON THE EXECUTORS -- output may not even appear in your driver console
// in cluster mode; this is a common beginner surprise

nums.saveAsTextFile("hdfs:///output/numbers")
// produces MULTIPLE files: part-00000, part-00001, ... one per partition

val words = sc.parallelize(List("a", "b", "a", "c", "b", "a"))
println(words.countByValue())
// Map(a -> 3, b -> 2, c -> 1)
```

Action	Returns	What It Does

> ⚠ **collect() is a genuine production hazard**
>
> Calling `.collect()` on an RDD with billions of rows attempts to pull ALL of that data into the Driver's single JVM heap — this is one of the single most common causes of a Spark job crashing with an OutOfMemoryError in real production incidents. Prefer `.take(n)` for a safe preview, `.count()` for a size check, or writing results out with `.saveAsTextFile()` instead of collecting large results back to the Driver.

4.2 Narrow vs. Wide Transformations, and the Shuffle

Not all transformations from Section 3 cost the same. This distinction determines exactly where Spark's DAG scheduler draws STAGE boundaries, and is the single most important performance concept in this unit.



Narrow vs. Wide Transformations — Where the Shuffle Comes From

- Narrow transformations (map, filter, flatMap, union) — each output partition depends on exactly one input partition. Spark can execute these entirely within one Executor, with zero network transfer.
- Wide transformations (groupByKey, reduceByKey, join, distinct, repartition) — an output partition may need data originally sitting in ANY input partition, forcing a SHUFFLE: data is written to disk, transferred across the network, and re-read on its destination node.

```
val rdd = sc.textFile("hdfs:///data/orders.csv")

val stage1 = rdd.filter(_.nonEmpty)         // narrow -- same stage
                .map(_.split(","))          // narrow -- same stage
val stage2 = stage1.map(f => (f(2), 1))     // narrow -- still same stage
                   .reduceByKey(_ + _)      // WIDE -- NEW STAGE starts here

// Confirm this visually
println(stage2.toDebugString)
// Look for the '+-(n)' markers -- each one is a shuffle / stage boundary
```

4.3 Caching and Persistence Levels — a Closer Look

Section 1.5 introduced .cache() and .persist(); here is the complete set of storage levels available, and how to choose between them.

StorageLevel	Behavior
MEMORY_ONLY	Store as deserialized Java objects in RAM. Fastest, but recomputes from lineage if it doesn't fit
MEMORY_AND_DISK	Spill to disk if RAM runs out, instead of recomputing — trades some speed for reliability
MEMORY_ONLY_SER	Store as SERIALIZED bytes in RAM — more compact, slightly slower to read
DISK_ONLY	Store only on local disk — for data too large for memory, still faster than full recomputation
MEMORY_AND_DISK_2	Like MEMORY_AND_DISK, but replicates each partition on 2 nodes for extra fault tolerance

```
import org.apache.spark.storage.StorageLevel

val rdd = sc.textFile("hdfs:///data/big.csv").map(expensiveParse)

rdd.persist(StorageLevel.MEMORY_AND_DISK_SER)

// Check whether an RDD is currently cached
println(rdd.getStorageLevel)

// ALWAYS release cached data once you're done with it
rdd.unpersist()
```

> **A Toll Bridge, Not a Free Highway**

StorageLevel	Behavior

A narrow transformation is driving on a road entirely within your own neighborhood -- no tolls, no checkpoints, you just go. A wide transformation is crossing a toll bridge to a DIFFERENT neighborhood -- every car has to stop, pay a toll (write to disk), wait in a queue (network transfer), and get checked in on the other side (read + merge). A job with ten wide transformations in a row is ten toll bridges, one after another -- and Spark's whole DAG-optimization effort (recall Unit II's Catalyst-adjacent theme of query planning) is largely about minimizing how many of those bridges a job actually has to cross.

Where this shows up in real performance tuning work

"Why is my Spark job slow" investigations overwhelmingly trace back to unnecessary or poorly-ordered wide transformations — an accidental `groupByKey` where `reduceByKey` would suffice, or a `join` performed before a `filter` that could have shrunk the data first. Experienced Spark engineers habitually check the number of shuffle stages in the Spark UI's DAG visualization (the same port-4040 UI from Unit I) as their first diagnostic step on any slow job.

Check yourself — Section 4

Q1. Why is `.collect()` dangerous on a large RDD, and what's a safer alternative for previewing data?

Q2. In your own words, explain why a wide transformation requires a shuffle but a narrow one doesn't.

Q3. When would you choose `MEMORY_AND_DISK` over `MEMORY_ONLY` for caching an RDD?

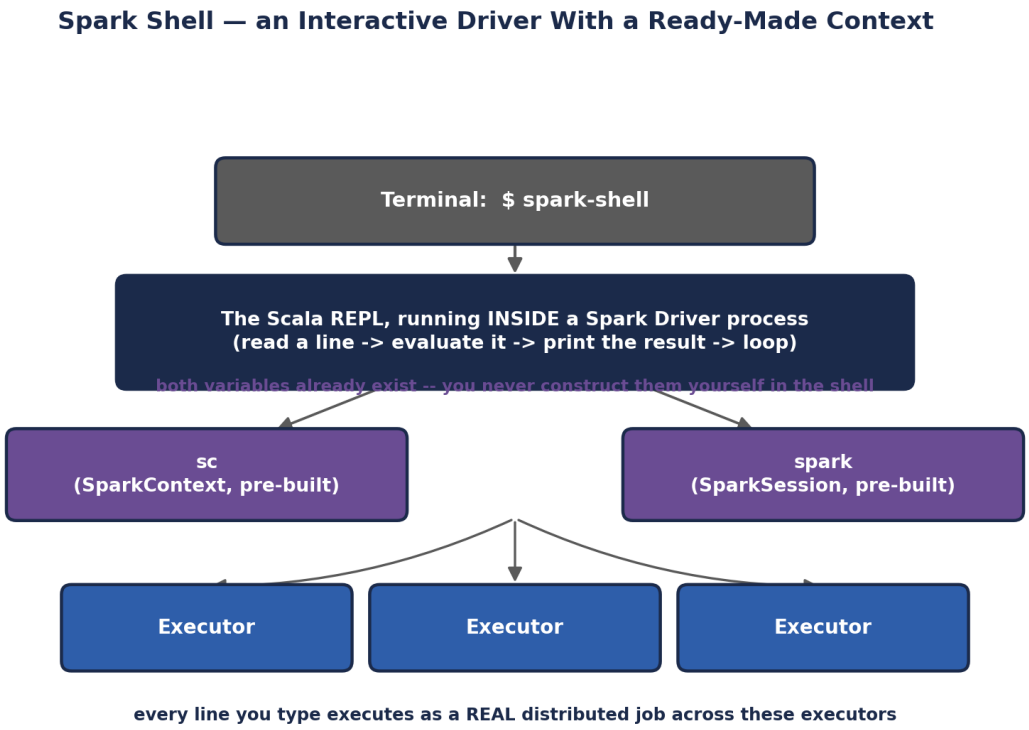
Hands-on Challenge:

Build a small RDD pipeline with at least two narrow transformations followed by one wide transformation (e.g., filter -> map -> reduceByKey), call `.toDebugString` before running anything, and identify exactly where the stage boundary appears in the output.

5. Invoking the Spark Shell

Unit I Section 5 walked through installing a Standalone Spark cluster. This section goes deeper into the interactive Scala shell itself -- the tool you'll actually use for most of this unit's hands-on exercises -- and precisely what makes it different from an ordinary Scala REPL.

5.1 Starting the Shell



Spark Shell — an Interactive Driver With a Ready-Made Context

```
# Launch against local mode (single JVM, simulates a small cluster) --
# perfect for the exercises in this unit
spark-shell

# Launch against your Unit I Standalone cluster specifically
spark-shell --master spark://<hostname>:7077

# Control resources explicitly
spark-shell --master spark://<hostname>:7077 \
            --executor-memory 2g \
            --total-executor-cores 4

# Pull in an extra library (e.g. a Kafka connector) for this session only
```

```
spark-shell --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0
```

The moment the shell finishes starting, it prints something like this, and it's worth reading carefully:

```
Spark context Web UI available at http://192.168.1.10:4040
Spark context available as 'sc' (master = local[*], app id = local-1234567890).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.0
      /_/

scala>
```

Two things in that banner matter directly: the port 4040 URL is exactly the Application UI from Unit I Section 5.5, live for this shell session specifically; and sc and spark are already constructed and ready to use -- you never write new SparkContext(...) yourself inside the shell.

5.2 Working in the Shell

```
scala> val rdd = sc.parallelize(1 to 10)
rdd: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[0] at parallelize

scala> rdd.filter(_ % 2 == 0).collect()
res0: Array[Int] = Array(2, 4, 6, 8, 10)

// Multi-line input: the shell waits for a complete expression
scala> val result = rdd
     |   .filter(_ % 2 == 0)
     |   .map(_ * 10)
     |   .collect()
result: Array[Int] = Array(20, 40, 60, 80, 100)

// :paste mode -- for pasting larger blocks of code cleanly,
// avoiding the REPL trying to (mis)evaluate each line individually
scala> :paste
// Entering paste mode (ctrl-D to finish)

def classify(n: Int): String =
  if (n % 2 == 0) "even" else "odd"

// (ctrl-D here)

// Checking a value's inferred type WITHOUT running it (Unit II Section 4)
```

```
scala> :type rdd.filter(_ > 5)
org.apache.spark.rdd.RDD[Int]

// Exiting
scala> :quit
```

5.3 PySpark's Equivalent Shell

Worth knowing even in a Scala-focused unit: the exact same interactive experience exists for Python, via pyspark, with sc and spark pre-built identically.

```
$ pyspark --master spark://<hostname>:7077

>>> rdd = sc.parallelize(range(1, 11))
>>> rdd.filter(lambda x: x % 2 == 0).collect()
[2, 4, 6, 8, 10]
```

A Test Kitchen With the Pantry Already Stocked

Writing a standalone Spark application (spark-submit) is like cooking in your own kitchen -- you have to first go shopping (import libraries, construct a SparkContext, configure everything) before you can cook a single dish. The Spark shell is a test kitchen where the pantry is ALREADY fully stocked the moment you walk in -- sc and spark sitting right there on the counter, ready to use -- specifically so you can experiment with one recipe at a time, tasting as you go, without any setup ceremony getting in the way.

Why the shell matters beyond just "learning"

Experienced Spark engineers use spark-shell (or its notebook cousins — Databricks notebooks, Jupyter with a PySpark kernel, Zeppelin) constantly in real production work: exploring an unfamiliar dataset's schema before writing a full pipeline, debugging why a specific transformation produces unexpected output by running it interactively step-by-step, or quickly checking a hypothesis against real cluster data before committing it to a scheduled job. The shell isn't a toy — it's a core professional tool.

Check yourself — Section 5

Q1. What are `sc` and `spark` inside the Spark shell, and why don't you construct them yourself?

Q2. What port does the shell's own Application UI run on, and what would you expect to see there?

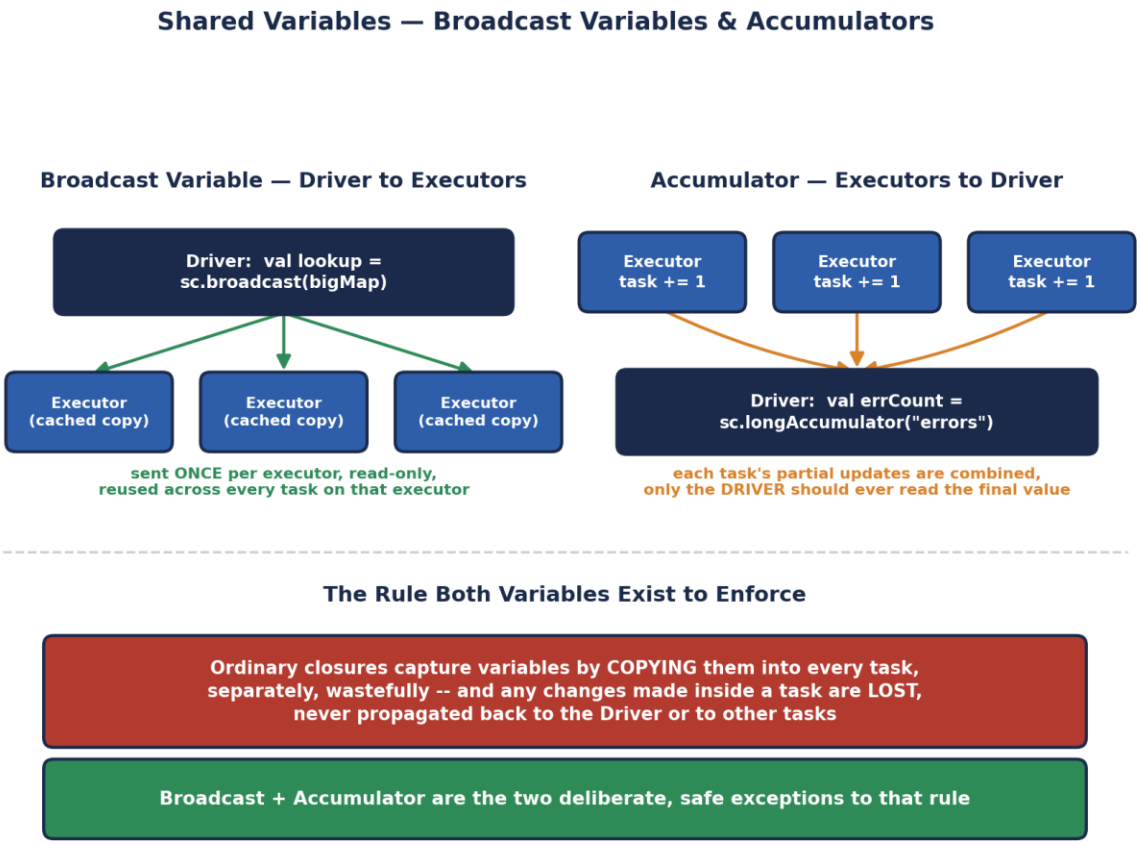
Q3. When would you use `:paste` mode instead of typing code directly at the `scala>` prompt?

Hands-on Challenge:

Start a Spark shell (local mode is fine), create an RDD from a list of at least 10 numbers, run a multi-line chained transformation using the `scala> ... | ...` continuation syntax, and confirm the result. Then open the port 4040 Application UI in a browser and identify the job corresponding to the action you just ran.

6. Shared Variables

By default, when a Spark transformation's function (a closure, in Unit II Section 6.2's terms) references an outside variable, Spark serializes a SEPARATE COPY of that variable into every single task. This is usually fine for small values, but breaks down in two specific, common situations -- which is exactly why Spark provides two special-purpose shared variable types.



Shared Variables — Broadcast Variables & Accumulators

6.1 The Problem With Ordinary Closures

```
val lookupTable = Map("US" -> "United States", "IN" -> "India", "UK" -> "United
Kingdom")

val countryCodes = sc.parallelize(List("US", "IN", "US", "UK", "IN"))

// This WORKS, but is wasteful: lookupTable is serialized and sent
// SEPARATELY with EVERY SINGLE TASK, even though every task on the
// same executor is sending an identical copy
val expanded = countryCodes.map(code => lookupTable(code))
```

6.2 Broadcast Variables — Efficient Read-Only Sharing

A broadcast variable is sent to each EXECUTOR exactly once, cached there, and reused by every task running on that executor -- rather than being re-sent for every individual task. For a large lookup table used across thousands of tasks, this is a dramatic reduction in network traffic.

```
val lookupTable = Map("US" -> "United States", "IN" -> "India", "UK" -> "United
Kingdom")

// Wrap it in a broadcast variable ONCE
val broadcastLookup = sc.broadcast(lookupTable)

val countryCodes = sc.parallelize(List("US", "IN", "US", "UK", "IN"))

// Access the underlying value with .value
val expanded = countryCodes.map(code => broadcastLookup.value(code))
println(expanded.collect().toList)
// List(United States, India, United States, United Kingdom, India)

// Release the cached copies on executors once you're done
broadcastLookup.unpersist()
broadcastLookup.destroy()   // fully removes it -- cannot be used again after
this
```

> ⚠ **Broadcast variables must be read-only**
>
> A broadcast variable's `.value` should never be mutated inside a task — each executor holds its own local copy, so a mutation on one executor would silently NOT be visible on any other executor, producing inconsistent, hard-to-debug results. Broadcast variables exist specifically for read-only, reference-style data (lookup tables, trained model coefficients, configuration).

6.3 Accumulators — Safe Aggregation Back to the Driver

An accumulator is a variable that tasks running on EXECUTORS can only ADD to -- never read a running total from -- and whose final combined value is reliably visible back on the DRIVER once an action completes. This is the safe alternative to trying to use an ordinary var for counting across a distributed job, which -- because of the closure-copying problem above -- simply would not work: each task would silently increment its own local copy, and the Driver would never see any of those increments.

```
val data = sc.parallelize(List("10", "20", "bad", "30", "oops", "40"))

val errorCount = sc.longAccumulator("parseErrors")

val parsed = data.map { s =>
  try {
```

```
    s.toInt
  } catch {
    case _: NumberFormatException =>
      errorCount.add(1)   // safely combined across ALL executors
      0
  }
}

val validSum = parsed.filter(_ != 0).reduce(_ + _)   // triggers the computation

println(s"Sum: $validSum")          // Sum: 100
println(s"Errors: ${errorCount.value}")   // Errors: 2

// Custom accumulators are also possible via AccumulatorV2 for non-numeric types
```

⚠ The one rule that trips almost everyone up at least once

Only read an accumulator's `.value` on the DRIVER, and only AFTER an action has actually run — reading it from within a transformation (on an Executor) is unsupported and gives unreliable results. It's also worth knowing that if an accumulator update happens inside a transformation that gets RECOMPUTED (Section 1.1's lineage-based recovery, or simply because the RDD wasn't cached and is used twice), that update can be counted MORE THAN ONCE — a subtle correctness trap. Prefer using accumulators for monitoring/debugging counters, not for business-critical calculations.

A Company Memo vs. a Shared Suggestion Box

A broadcast variable is a company-wide memo printed and distributed ONCE to every department's mailroom (each Executor), then referenced by every employee (task) in that department without needing a fresh copy mailed to each individual desk. An accumulator is a shared suggestion box bolted to the wall in the main lobby -- any employee, in any department, can walk up and DROP IN a suggestion (add to it), but nobody except management (the Driver) is allowed to open the box and read what's inside, and only after the collection period has genuinely ended (an action has completed).

Where these show up in real Spark applications

Broadcast variables are the standard mechanism for shipping a small-to-medium reference dataset (a country-code table, a trained model's coefficients, a denylist) to every node without paying a per-task serialization cost — and are exactly how Spark implements broadcast JOINS internally (joining a huge table against a small one by broadcasting the small side instead of shuffling both). Accumulators show up constantly as lightweight monitoring counters in production ETL jobs — counting malformed records, tracking how many rows failed a validation rule — surfaced afterward in a job's logs or metrics dashboard.

Check yourself — Section 6

Q1. Why is sending a large lookup table via an ordinary closure wasteful compared to a broadcast variable?

Q2. Why can't you just use an ordinary `var` on the Driver to count errors occurring inside a `map()` transformation across a cluster?

Q3. What's the specific risk of reading an accumulator's value from WITHIN a transformation rather than on the Driver after an action?

Hands-on Challenge:

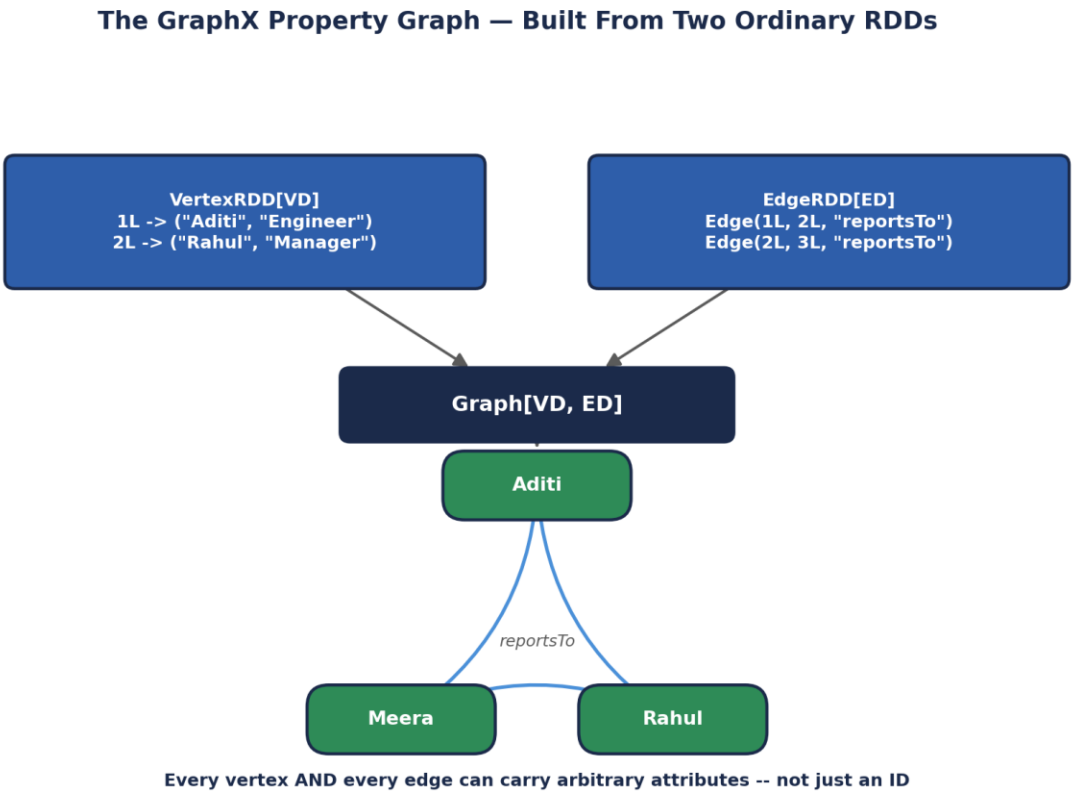
Create an RDD of at least 15 strings where some are valid integers and some are not. Use a broadcast variable to hold a Set of "banned" words to additionally filter out, and use an accumulator to count how many elements were rejected for EITHER reason (parse failure or banned word). Print the final valid sum and the total rejected count.

7. Introduction to Spark GraphX

Everything so far in this unit has treated data as ROWS — an RDD of independent elements or key-value pairs. GraphX introduces a fundamentally different shape of question: not "what does this row say," but "who or what is CONNECTED to whom, and how does influence, information, or structure flow across those connections." This is genuinely new territory with no direct equivalent among the RDD operations covered in Sections 1-6.

7.1 What a Graph Is, Formally

A graph consists of VERTICES (nodes — the "things": people, web pages, products) and EDGES (the relationships between them — follows, links-to, purchased). GraphX specifically works with PROPERTY GRAPHS, where both vertices and edges can carry arbitrary attached data, not just a bare identifier.



The GraphX Property Graph — Built From Two Ordinary RDDs

Notice the diagram's foundation: a Graph is built from exactly two RDDs you already know how to create from Section 2 — a VertexRDD[VD] (vertex ID paired with vertex data) and an EdgeRDD[ED] (source ID, destination ID, and edge data). GraphX doesn't introduce a new distributed data structure from scratch; it composes two ordinary, already-understood RDDs into a graph-shaped abstraction, inheriting every fault-tolerance and partitioning property from Section 1 automatically.

7.2 Building Your First Graph

```
import org.apache.spark.graphx._
import org.apache.spark.rdd.RDD

// Vertices: (vertex ID, (name, role))
val vertices: RDD[(VertexId, (String, String))] = sc.parallelize(Seq(
  (1L, ("Aditi", "Engineer")),
  (2L, ("Rahul", "Manager")),
  (3L, ("Meera", "Engineer")),
  (4L, ("Zoya", "Director"))
))

// Edges: Edge(sourceId, destId, edge attribute)
val edges: RDD[Edge[String]] = sc.parallelize(Seq(
  Edge(1L, 2L, "reportsTo"),
  Edge(2L, 4L, "reportsTo"),
  Edge(3L, 2L, "reportsTo")
))

val orgGraph: Graph[(String, String), String] = Graph(vertices, edges)

// Basic exploration
println(s"Vertices: ${orgGraph.numVertices}")   // 4
println(s"Edges: ${orgGraph.numEdges}")          // 3

orgGraph.vertices.collect().foreach {
  case (id, (name, role)) => println(s"$id: $name ($role)")
}

orgGraph.edges.collect().foreach {
  case Edge(src, dst, relationship) => println(s"$src --[$relationship]-->
$dst")
}
```

7.3 Why GraphX Instead of a Traditional Graph Database

A traditional graph database (Neo4j, for example) is optimized for fast, single-node-at-a-time traversal queries -- "find this specific person's friends-of-friends." GraphX is optimized for the opposite: running ONE algorithm across the ENTIRE graph in parallel across a cluster -- "compute the influence rank of every single node in a 500-million-edge social graph simultaneously." These are genuinely different workloads, and choosing between them is an architecture decision, not a matter of one being strictly better.

> **A City Map vs. a Satellite Weather System**
>
> *A traditional graph database is a detailed city map with a GPS app -- brilliant for answering "what's the fastest route from THIS house to THAT house," one query, one path, at a time. GraphX is a satellite weather system tracking the ENTIRE atmosphere at once -- it doesn't care about one specific path*

between two points; it's built to compute a value (temperature, pressure, in GraphX's case something like influence or connectivity) for every single point on the map SIMULTANEOUSLY, in one coordinated distributed pass.

Real-world graph-shaped problems

Social networks (who influences whom, community detection), recommendation systems ("people who bought X also bought Y" is a graph traversal), fraud detection (a ring of accounts transacting suspiciously with each other forms a detectable graph pattern), web search (PageRank, GraphX's most famous built-in algorithm, is literally how early Google ranked pages by treating the web as one enormous graph), and telecom network analysis (tracing outage propagation through a physical network graph) are all naturally graph-shaped problems that a purely row-based RDD or DataFrame view struggles to express cleanly.

Check yourself — Section 7

Q1. What two RDDs is every GraphX Graph built from, and what does each one contain?

Q2. What's the key architectural difference between GraphX and a traditional graph database like Neo4j?

Q3. Name one real-world problem that is naturally graph-shaped, and explain briefly why a plain table of rows would struggle to express it.

Hands-on Challenge:

Build your own small property graph representing a social network of at least 5 people (vertices with a name attribute) and at least 6 "follows" relationships (edges), then print `numVertices`, `numEdges`, and iterate over the edges to print each relationship in a readable sentence.

8. Spark GraphX Features

This section catalogs GraphX's defining features -- the specific design decisions that make it fit naturally into everything covered so far in this unit, rather than being a bolted-on, unrelated library.

8.1 Resilient Distributed Property Graph

Directly inheriting Section 1's RDD features by construction: a GraphX Graph is immutable, partitioned, and lineage-tracked, because it's built from two ordinary RDDs. A lost partition of vertices or edges recovers through recomputation from lineage exactly as described in Section 1.1 -- GraphX adds no new fault-tolerance mechanism of its own; it simply inherits Spark Core's.

8.2 Flexible, Composable API

GraphX operations return NEW graphs (or new RDDs), following the same immutability discipline as every RDD transformation in Sections 3-4. This means graph operations chain together exactly the way RDD transformations do, and can be freely mixed with ordinary RDD operations at any point.

```
// A Graph exposes its underlying vertices and edges AS ORDINARY RDDs --
// every Section 3/4 transformation and action works on them directly
val highLevelEmployees = orgGraph.vertices.filter {
  case (id, (name, role)) => role == "Director" || role == "Manager"
}
println(highLevelEmployees.collect().toList)

// degrees: how many edges touch each vertex -- returns an ordinary VertexRDD
val degrees = orgGraph.degrees
degrees.collect().foreach { case (id, degree) => println(s"Vertex $id has degree
$degree") }
```

8.3 A Built-In Library of Graph Algorithms

GraphX ships with several classic graph algorithms already implemented and optimized for distributed execution, so common analyses don't need to be hand-written from scratch.

Algorithm	What It Computes
`PageRank`	A relative "importance" score for each vertex, based on how many (and how important) other vertices link to it
`Connected Components`	Groups vertices into clusters where every vertex can reach every other vertex in the same cluster

Algorithm	What It Computes
`Triangle Counting`	Counts how many triangles (3-way mutual connections) each vertex participates in — a common measure of local clustering
`Shortest Paths`	The minimum number of hops from a set of source vertices to every other vertex
`Label Propagation (LPA)`	A fast, heuristic community-detection algorithm — vertices adopt the most common label among their neighbors, iteratively

```
// PageRank -- GraphX's most famous built-in algorithm
val ranks = orgGraph.pageRank(tol = 0.0001).vertices
ranks.collect().sortBy(-_._2).foreach {
  case (id, rank) => println(f"Vertex $id%d has rank $rank%.4f")
}

// Connected components -- who's reachable from whom
val components = orgGraph.connectedComponents().vertices
components.collect().foreach {
  case (id, componentId) => println(s"Vertex $id belongs to component
$componentId")
}

// Triangle counting -- requires the graph to be first "canonicalized"
val triCounts = orgGraph.partitionBy(PartitionStrategy.RandomVertexCut)
                        .triangleCount()
                        .vertices
```

8.4 Efficient Distributed Execution — Vertex-Cut Partitioning

Naively partitioning a graph by splitting up EDGES can leave a single highly-connected vertex (a celebrity account with millions of followers, for instance) scattered across nearly every partition, creating a severe load imbalance. GraphX instead defaults to VERTEX-CUT partitioning: edges are distributed across partitions such that each vertex's data, even a highly-connected one, is replicated only as needed, keeping computation balanced across the cluster.

> **A Well-Organized Group Photo, Not a Scattered Crowd**
>
> *Imagine photographing a party where one extremely popular guest (a high-degree vertex) is talking to a hundred different small groups scattered across the room. A naive photographer (edge-cut partitioning) tries to capture every conversation that guest is having as a SEPARATE photo, forcing that one guest to physically run between a hundred different corners of the room. GraphX's vertex-cut approach instead is smart about which conversations get grouped into which photos, so that popular guest's information is*

efficiently shared exactly where it's needed without wastefully duplicating effort or creating one massively overloaded corner of the room.

Why this partitioning detail is not just internal trivia

Real-world graphs (social networks, web link graphs) follow a power-law degree distribution — a small number of vertices have enormous numbers of connections, while most have very few. Naive partitioning strategies handle this poorly, creating severe stragglers (a few tasks taking dramatically longer than the rest, echoing the data-skew concept from earlier units). GraphX's default vertex-cut strategy exists specifically because this power-law pattern is the REALISTIC shape of graph data in production, not an edge case.

Check yourself — Section 8

Q1. Why does a GraphX Graph automatically get fault tolerance, without GraphX implementing its own separate recovery mechanism?

Q2. Name three built-in GraphX algorithms and what each one computes, in one phrase each.

Q3. Why does GraphX default to vertex-cut partitioning instead of a simpler edge-cut approach?

Hands-on Challenge:

Using the social network graph you built in Section 7's exercise, run `.pageRank()` on it and print the results sorted from highest to lowest rank. Then run `.connectedComponents()` and check whether your graph forms one single connected component or several separate ones.

9. Spark GraphX Operations

Beyond the built-in algorithms from Section 8.3, GraphX exposes a set of general-purpose operators for building CUSTOM graph computations -- structural operators that reshape a graph, join operators that merge in outside data, and the Pregel API for writing your own iterative, message-passing algorithms.

9.1 Structural Operators

Operator	What It Does
`reverse`	Returns a new graph with every edge's direction flipped
`subgraph`	Returns a new graph restricted to vertices/edges satisfying given predicates
`mask`	Restricts a graph to only the vertices/edges also present in another graph
`groupEdges`	Merges parallel edges (multiple edges between the same pair of vertices) using a combining function

```
// reverse -- flip every "reportsTo" edge into "manages"
val managementGraph = orgGraph.reverse

// subgraph -- restrict to just Engineers and the edges between them
val engineersOnly = orgGraph.subgraph(
  vpred = (id, attr) => attr._2 == "Engineer"
)
println(s"Engineers only: ${engineersOnly.numVertices} vertices,
${engineersOnly.numEdges} edges")

// groupEdges -- collapse duplicate edges, e.g. multiple "purchased" events
// between the same customer and product, into one edge with a combined count
val purchaseGraph: Graph[String, Int] = Graph(vertices.map(v => (v._1,
v._2._1)),
  sc.parallelize(Seq(Edge(1L, 2L, 1), Edge(1L, 2L, 1), Edge(1L, 2L, 1))))
val grouped = purchaseGraph.groupEdges((countA, countB) => countA + countB)
// the three separate weight-1 edges become ONE edge with weight 3
```

9.2 Join Operators — Bringing in Outside Data

Join operators let you enrich a graph's vertices with data computed SEPARATELY (often the output of an algorithm from Section 8.3), without rebuilding the whole graph from scratch.

```
// joinVertices -- update EXISTING vertex attributes using outside data,
```

```
// dropping any vertex not present in the join source
val ranks = orgGraph.pageRank(0.0001).vertices   // RDD[(VertexId, Double)]

val rankedGraph = orgGraph.outerJoinVertices(ranks) {
  case (id, (name, role), rankOpt) =>
    (name, role, rankOpt.getOrElse(0.0))   // outer join -- keeps ALL original
vertices
}
rankedGraph.vertices.collect().foreach {
  case (id, (name, role, rank)) => println(f"$name ($role): $rank%.4f")
}
```

9.3 aggregateMessages — the Simpler, One-Hop Building Block

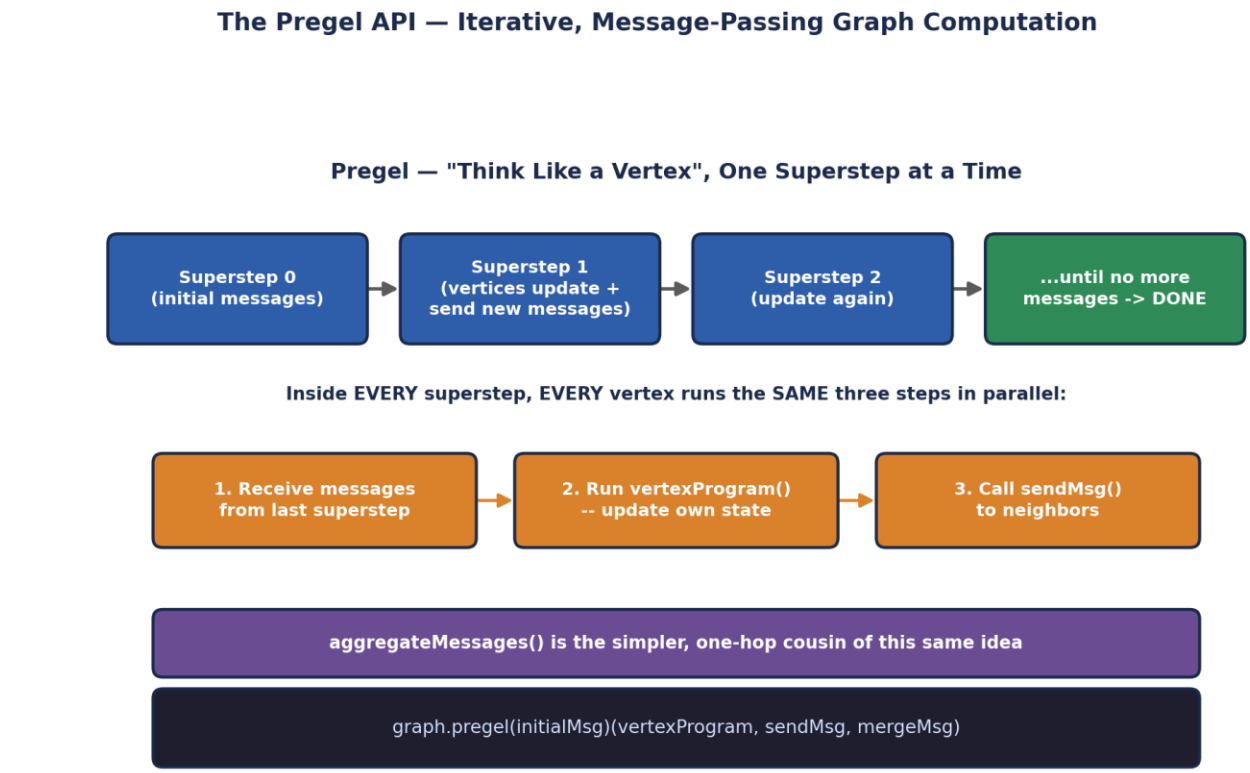
Before reaching for the full Pregel API, many graph computations only need ONE round of neighbor-to-neighbor communication -- exactly what aggregateMessages provides: every edge can send a message to its source and/or destination vertex, and those messages are combined per-vertex using a merge function.

```
// Compute the average "seniority score" of each vertex's neighbors
val seniority = sc.parallelize(Seq((1L, 2), (2L, 5), (3L, 2), (4L, 8)))
val scoredGraph = orgGraph.outerJoinVertices(seniority) {
  case (id, (name, role), scoreOpt) => (name, role, scoreOpt.getOrElse(0))
}

val neighborAverages = scoredGraph.aggregateMessages[(Int, Int)](
  triplet => {
    // sendMsg: for every edge, tell the DESTINATION vertex about the
    // SOURCE vertex's score, as a (sum, count) pair for later averaging
    triplet.sendToDst((triplet.srcAttr._3, 1))
  },
  (a, b) => (a._1 + b._1, a._2 + b._2)   // mergeMsg: combine all incoming
messages
)

val averaged = neighborAverages.mapValues { case (sum, count) => sum.toDouble /
count }
averaged.collect().foreach { case (id, avg) => println(f"Vertex $id%d neighbor
avg: $avg%.2f") }
```

9.4 The Pregel API — General Iterative Graph Computation

Named after Google's own internal graph-processing system, Pregel is GraphX's API for writing arbitrary iterative, message-passing algorithms -- in fact, PageRank and Connected Components from Section 8.3 are themselves implemented internally using Pregel. It runs in SUPERSTEPS: rounds where every vertex, in parallel, processes

incoming messages, updates its own state, and sends new messages to its neighbors -- repeating until no more messages are sent.



The Pregel API — Iterative, Message-Passing Graph Computation

```
// A simplified single-source shortest-path computation using Pregel
val sourceId: VertexId = 1L

val initialGraph = orgGraph.mapVertices((id, _) =>
  if (id == sourceId) 0.0 else Double.PositiveInfinity
)

val shortestPaths = initialGraph.pregel(Double.PositiveInfinity)(
  // vertexProgram: given my current distance and an incoming message,
  // update my own state to whichever is smaller
  (id, currentDist, newDist) => math.min(currentDist, newDist),

  // sendMsg: propose "my distance + 1" to each neighbor
  triplet => {
    if (triplet.srcAttr + 1 < triplet.dstAttr) {
      Iterator((triplet.dstId, triplet.srcAttr + 1))
    } else {
      Iterator.empty   // no improvement -- send nothing, helps convergence
    }
  },
```

```
  // mergeMsg: if a vertex gets multiple proposals in one superstep, keep the
smallest
  (a, b) => math.min(a, b)
)

shortestPaths.vertices.collect().sortBy(_._1).foreach {
  case (id, dist) => println(s"Distance from $sourceId to $id: $dist")
}
```

⚠ A common source of confusion

Pregel's `sendMsg` runs PER EDGE (via the EdgeTriplet, which exposes both endpoints' current attributes), while `vertexProgram` runs PER VERTEX, combining that vertex's current state with whatever `mergeMsg` produced from all of its incoming messages that superstep. Mixing up which function operates on edges versus vertices is the single most common mistake when writing a first Pregel algorithm.

A Very Organized Game of Telephone

Ordinary "telephone" is chaotic and sequential -- one whisper at a time, down a line. Pregel is a perfectly organized, synchronized version: in ROUND 1, every single person in the room whispers to their immediate neighbors SIMULTANEOUSLY. Everyone then updates what they personally know based on what they just heard. In ROUND 2, everyone whispers again, based on their NEW updated knowledge. This repeats, round after round, until an entire round passes with nobody having anything new to whisper -- at which point the game is over, and (for something like shortest-path) everyone in the room now correctly knows their true distance from the original speaker.

Why Pregel-style computation matters at scale

Google's original Pregel paper existed to solve exactly the problem this section demonstrates: some graph algorithms are inherently iterative and don't fit a single MapReduce-style pass. Running PageRank the naive way — as a fixed sequence of separate MapReduce jobs, one per iteration — was exactly the kind of disk-bound, multi-stage workload Unit I identified as MapReduce's weakness. GraphX's Pregel implementation, built on Spark's in-memory RDDs, keeps the graph's state resident in memory across supersteps, turning what would otherwise be a painfully slow multi-job MapReduce pipeline into a single, fast, in-memory iterative computation.

Check yourself — Section 9

Q1. What's the difference between `joinVertices` and `outerJoinVertices`, in terms of which vertices survive the join?

Q2. In Pregel, which function runs per-edge, and which runs per-vertex?

Q3. Why is Pregel-style iterative computation a better fit for algorithms like PageRank than a sequence of separate MapReduce jobs would be?

Hands-on Challenge:

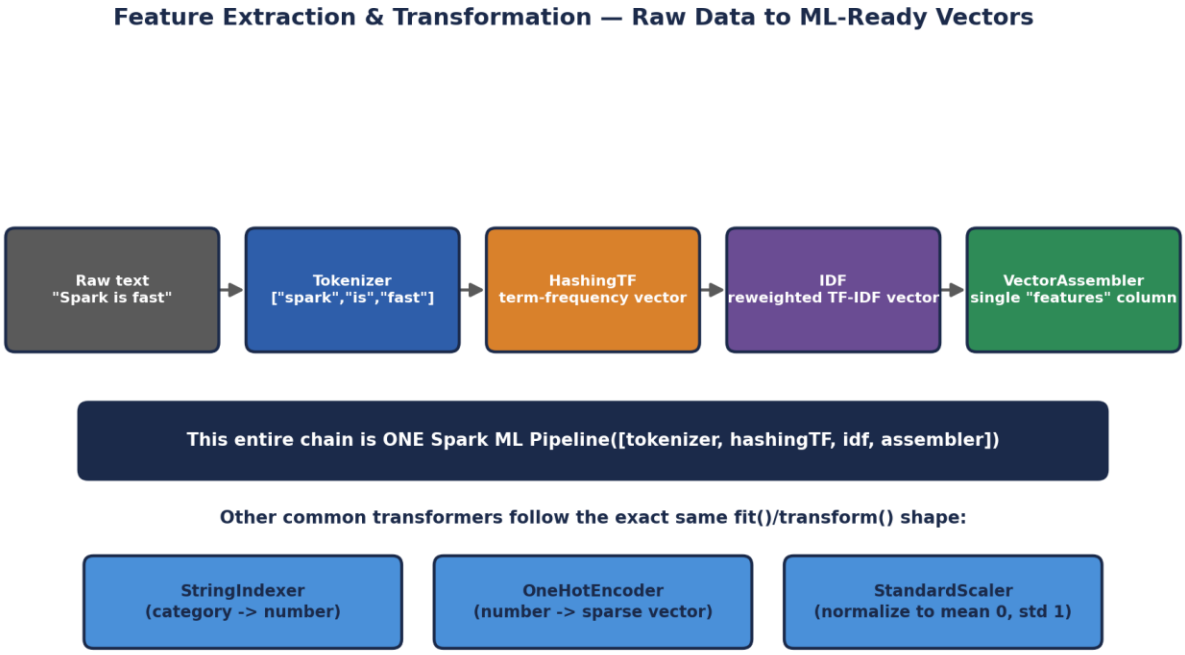
Using your Section 7 social-network graph, write a Pregel computation that finds each vertex's shortest "hop distance" from one chosen starting person, following the shortest-path pattern shown above. Print the final distance from your chosen source to every other vertex in the graph.

10. Feature Extraction and Transformation

This closing section shifts from graphs back to a different kind of preparation problem: turning raw, messy, real-world data (text, categories, differently-scaled numbers) into the clean numeric VECTORS that machine learning algorithms actually require as input. This lives in Spark's org.apache.spark.ml.feature package and follows the exact Estimator/Transformer/fit()/transform() pattern that later units on Spark MLlib will build on directly.

10.1 Why Raw Data Needs Transformation

A machine learning algorithm ultimately operates on vectors of numbers -- it has no native understanding of a sentence of English text, a category like "Electronics", or the fact that a $50,000 salary and a 0.02 interest rate exist on wildly different numeric scales. Feature extraction and transformation is the deliberate, systematic process of closing that gap.



Feature Extraction & Transformation — Raw Data to ML-Ready Vectors

10.2 Text Feature Extraction — Tokenizer, HashingTF, and IDF

```
import org.apache.spark.ml.feature.{Tokenizer, HashingTF, IDF}
import org.apache.spark.sql.SparkSession

val spark = SparkSession.builder.appName("FeatureDemo").getOrCreate()
import spark.implicits._

val sentences = Seq(
```

```
  (0, "Spark is fast and general purpose"),
  (1, "Scala is elegant and expressive"),
  (2, "Spark is written in Scala")
).toDF("id", "sentence")

// Step 1: Tokenizer -- split each sentence into individual words
val tokenizer = new Tokenizer().setInputCol("sentence").setOutputCol("words")
val wordsData = tokenizer.transform(sentences)
wordsData.select("words").show(false)
// [spark, is, fast, and, general, purpose]
// [scala, is, elegant, and, expressive]
// [spark, is, written, in, scala]

// Step 2: HashingTF -- convert words into a fixed-size term-frequency vector
val hashingTF = new HashingTF()
  .setInputCol("words").setOutputCol("rawFeatures").setNumFeatures(1000)
val featurizedData = hashingTF.transform(wordsData)

// Step 3: IDF -- down-weight words that appear in EVERY document (like "is"),
// and up-weight words that are more DISTINCTIVE
val idf = new IDF().setInputCol("rawFeatures").setOutputCol("features")
val idfModel = idf.fit(featurizedData)   // fit() -- an ESTIMATOR, learns from
the data
val rescaledData = idfModel.transform(featurizedData)   // transform() --
applies it
rescaledData.select("id", "features").show(false)
```

10.3 Categorical Feature Encoding — StringIndexer and OneHotEncoder

Categorical text values ("Engineering", "Sales", "Marketing") have no inherent numeric ordering, but algorithms need numbers -- StringIndexer and OneHotEncoder together convert a category into a form that doesn't accidentally imply a false ordering (e.g., that "Marketing" > "Engineering" numerically, which a raw index alone would wrongly suggest to many algorithms).

```
import org.apache.spark.ml.feature.{StringIndexer, OneHotEncoder}

val employees = Seq(
  ("Aditi", "Engineering"), ("Rahul", "Sales"),
  ("Meera", "Engineering"), ("Zoya", "Marketing")
).toDF("name", "department")

// StringIndexer: "Engineering" -> 0.0, "Sales" -> 1.0, "Marketing" -> 2.0
// (ordered by FREQUENCY, most common category gets index 0)
val indexer = new StringIndexer()
  .setInputCol("department").setOutputCol("departmentIndex")
val indexed = indexer.fit(employees).transform(employees)
```

```
// OneHotEncoder: turns that single number into a SPARSE VECTOR where
// exactly one position is "hot" (1), avoiding any false numeric ordering
val encoder = new OneHotEncoder()
  .setInputCol("departmentIndex").setOutputCol("departmentVec")
val encoded = encoder.fit(indexed).transform(indexed)
encoded.select("name", "department", "departmentVec").show(false)
```

10.4 Numeric Scaling — StandardScaler

Features on wildly different numeric scales (a salary in the tens of thousands vs. a years-of-experience count under 40) can cause many ML algorithms to implicitly treat the larger-magnitude feature as more important, regardless of its actual predictive value. StandardScaler rescales every feature to have a mean of 0 and a standard deviation of 1, putting them on equal footing.

```
import org.apache.spark.ml.feature.{VectorAssembler, StandardScaler}

val raw = Seq(
  ("Aditi", 85000.0, 3.0), ("Rahul", 72000.0, 6.0),
  ("Meera", 91000.0, 2.0), ("Zoya", 130000.0, 10.0)
).toDF("name", "salary", "yearsExperience")

// First, assemble the raw numeric columns into ONE vector column
// (VectorAssembler -- the same operator from the pipeline diagram above)
val assembler = new VectorAssembler()
  .setInputCols(Array("salary", "yearsExperience")).setOutputCol("rawFeatures")
val assembled = assembler.transform(raw)

val scaler = new StandardScaler()
  .setInputCol("rawFeatures").setOutputCol("scaledFeatures")
  .setWithMean(true).setWithStd(true)
val scalerModel = scaler.fit(assembled)
val scaled = scalerModel.transform(assembled)
scaled.select("name", "scaledFeatures").show(false)
```

10.5 Chaining It All Into One Pipeline

Exactly as the diagram above shows, every one of these transformers can be chained into a single Pipeline object -- the same Pipeline/Estimator/Transformer pattern later units will use throughout Spark MLlib model training.

```
import org.apache.spark.ml.Pipeline

val pipeline = new Pipeline().setStages(Array(
  tokenizer, hashingTF, idf
```

```
))

val pipelineModel = pipeline.fit(sentences)   // ONE fit() call runs the whole
chain
val finalFeatures = pipelineModel.transform(sentences)
finalFeatures.select("id", "features").show(false)

// The fitted pipeline can be SAVED and reused later on new, unseen data --
// exactly the same feature engineering steps, guaranteed identical
pipelineModel.write.overwrite().save("hdfs:///models/text-features-pipeline")
```

A Restaurant's Standardized Prep Station

Raw ingredients arrive at a restaurant in wildly inconsistent forms -- whole vegetables, unlabeled sacks, a delivery in kilograms next to one in pounds. A feature pipeline is the restaurant's standardized prep station: every ingredient, no matter how it arrived, goes through the SAME sequence of stations -- washed, chopped to a consistent size, weighed on the SAME scale in the SAME units -- before it's allowed anywhere near the actual cooking (the ML algorithm). A chef (the model) that only knows how to work with perfectly diced, uniformly measured ingredients would be helpless facing a whole unwashed pumpkin -- exactly as helpless as an ML algorithm facing a raw sentence of English text or an un-scaled salary column.

Why this is one of the highest-leverage skills in applied ML

In real industry practice, feature engineering — not algorithm selection — is consistently cited by practicing data scientists as the single factor most responsible for a model's real-world performance. A mediocre algorithm fed well-engineered features routinely outperforms a sophisticated algorithm fed raw, unprocessed data. Because Spark's feature transformers operate on distributed DataFrames using the exact same fit()/transform() Estimator pattern as MLlib's actual models (a direct preview of upcoming units), the feature engineering skills in this section transfer immediately and directly into full model-training pipelines.

Check yourself — Section 10

Q1. Why can't a machine learning algorithm work directly with a raw sentence of text or a category like "Marketing"?

Q2. Why does OneHotEncoder exist — what problem would a raw StringIndexer output alone create if fed directly into most ML algorithms?

Q3. Why is StandardScaler important when a dataset has features on very different numeric scales (like salary vs. years of experience)?

Hands-on Challenge:

Using a small DataFrame of your own design with at least one text column, one categorical column, and one or two numeric columns, build a single Pipeline chaining a Tokenizer + HashingTF for the text

column, a StringIndexer + OneHotEncoder for the category, and a VectorAssembler + StandardScaler for the numeric columns. Fit the pipeline and show the final transformed output.

Unit III Review

Quick Review Quiz

Three questions covering the full breadth of this unit. Try to answer each from memory before checking against the sections above.

1. Q1. Why can an RDD recover from a lost partition without ever having stored a replicated backup copy of it?
2. Q2. Why is `reduceByKey` generally preferred over `groupByKey`, and what does that preference have in common with a shuffle in general?
3. Q3. In GraphX's Pregel API, what happens in each superstep, and when does the computation stop?

> **Answer key (cover before peeking)**
>
> A1: Spark tracks the LINEAGE of transformations that produced the RDD. If a partition is lost, Spark simply re-runs the recorded chain of transformations for just that partition, using the same coarse-grained, deterministic transformations from Section 1.6 — no replicated backup was ever needed.
>
> A2: `reduceByKey` combines values LOCALLY on each partition before shuffling only the smaller partial results, while `groupByKey` shuffles every raw value across the network first. Both this and the general cost of a shuffle (Section 4.2) come down to the same principle: minimize data crossing the network, because network and disk I/O are the most expensive resources in a distributed system.
>
> A3: In each superstep, every vertex, in parallel, receives messages from the previous superstep, updates its own state via vertexProgram, and sends new messages to neighbors via sendMsg. The computation stops once a full superstep passes with no messages sent at all.

Hands-On Exercise

Complete, end to end, using spark-shell (Section 5) against either local mode or your Unit I Standalone cluster:

4. Load a text file of your choosing with `sc.textFile()` (Section 2.2), or construct a representative RDD with `sc.parallelize()` if no file is available, containing at least 50 lines/records.
5. Build a word-count pipeline using `flatMap`, `map`, and `reduceByKey` (Section 3), and cache the resulting RDD (Section 1.5) before running two different actions against it.
6. Call `.toDebugString` on your final RDD and identify exactly where the narrow-to-wide transition (the shuffle boundary) occurs (Section 4.2).
7. Use a broadcast variable to hold a small set of "stop words" (like "the", "is", "a") to exclude from your word count, and an accumulator to count how many stop words were filtered out (Section 6).
8. Separately, build a small property graph (Section 7) representing any network you choose (colleagues, characters in a book, cities connected by roads), run PageRank and Connected Components on it (Section 8), and print the results sorted by rank.

9. Finally, using a small DataFrame with at least one text and one numeric column, build a feature-extraction Pipeline (Section 10) combining a Tokenizer/HashingTF chain with a StandardScaler, and show the final transformed output.

> ⚠ **What this exercise is really testing**
>
> This single exercise deliberately exercises every syllabus topic in this unit at once: creating and transforming RDDs (Sections 2-4), the interactive shell workflow (Section 5), safe shared state (Section 6), and both halves of the GraphX + feature engineering material (Sections 7-10) — giving you one continuous, cumulative proof that you can build a real, working Spark application end to end rather than just recognizing each concept in isolation.